

cute 之 Hopper MBarrier

[Hopper架构](#)上提供了更高效的计算单元和数据搬运单元，通过数据依赖协调这两个单元的执行进度十分重要，为此Hopper架构上提供了MBarrier功能单元，本文将重点介绍MBarrier相关的功能，数据表示和状态转移，并以实验的形式呈现该状态转移。文章结构上，首先回顾了NVidia GPU上的Tensor Core和数据搬运单元，由此引入了GPU上的同步单元，然后重点介绍了MBarrier的核心功能、数据结构表示和其状态转移，最后我们给出了文中讲解的示例代码并对文章进行了总结。

计算和数据搬运的桥梁

在高性能处理器架构体系中，有两个核心点，一个核心是计算，另一个核心是数据搬运。处理器的推陈出新始终是在围绕更高效的计算核心和更高效的数据搬运引擎在演进。

NVidia GPU的演进自然也在这个范式之内，如图1所示，从[Volta架构](#)开始，GPU的处理器单元SM（Stream Multiprocessor）上装配了Tensor Core来提高计算能力，尤其是矩阵计算能力，从[Turing架构](#)开始提供了`ldmatrix`指令以提升共享内存（shared memory）向寄存器（register）进行数据搬运的能力，[Ampere架构](#)上提供了`cp.async`指令实现全局内存（global memory）到共享内存异步数据搬运的能力，在Hopper架构中针对计算部分，其装配了具有异步执行能力的WGMMA（WarpGroup Matrix Multiply-Accumulate）Tensor Core，针对数据搬运部分，其装配了独立的数据搬运TMA（Tensor Memory Accelerator）单元。

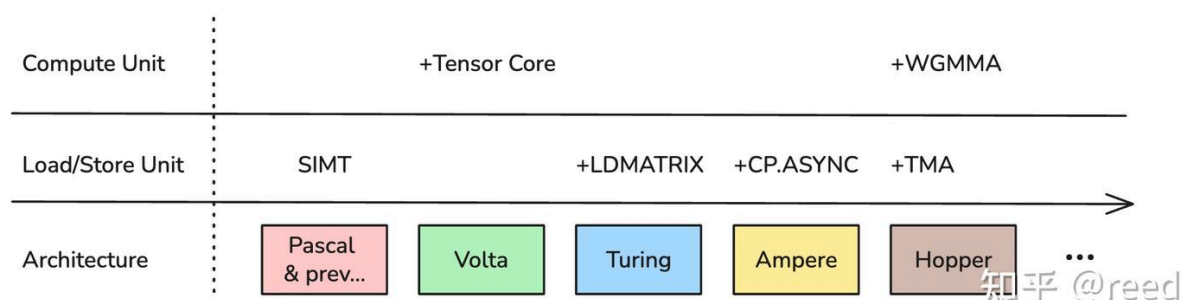


Figure-1. Compute and Load/Store unit envolution

Hopper架构之前的Tensor Core单元，在进行矩阵乘加计算时其指令的源操作数和目的操作数都是寄存器（如图2中mma指令），其指令执行周期也是固定的，Hopper架构的Tensor Core计算单元除了可以完成mma指令，还可以连续四个warp组成warp group共同完成更大规格的矩阵乘法，这是使用的指令为warp group层级的指令，即WGMMA指令，该指令逻辑上的输入矩阵A除了可以存储在寄存器上，还可以存储在共享内存上，B矩阵必须存储在共享内存上，C矩阵为输出矩阵，其存储在

寄存器上。不同于同步的MMA指令，该WGMMA指令是异步的，需要配合额外的commit和wait指令完成计算并且确保结果的可见性。由于其计算规格更大，并且多个warp协同完成，而且支持更低的输入精度（如FP8）其计算效率更高。

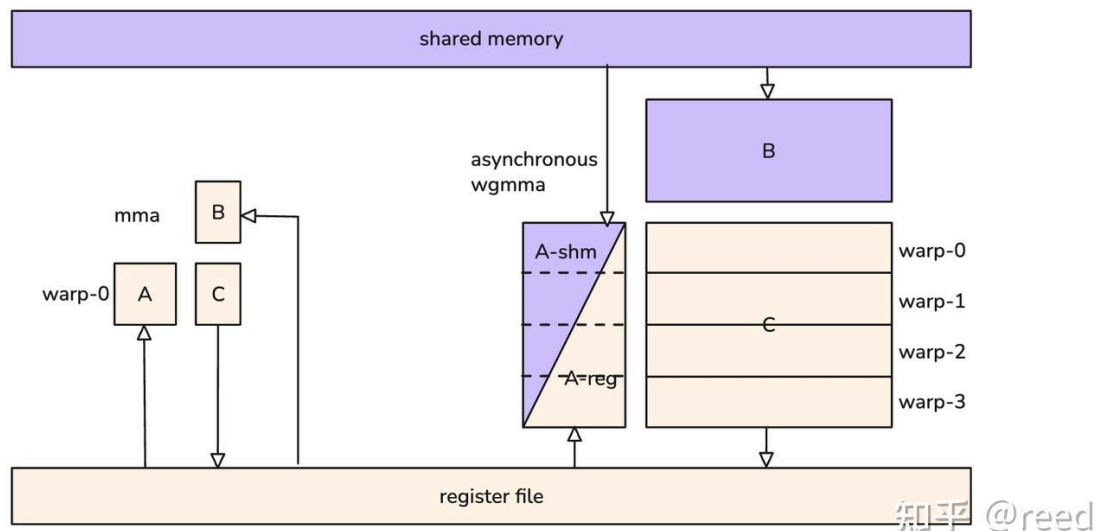


Figure-2. Hopper tensor core instruction operand source location

Hopper除了提供了高效的WGMMA能力完成矩阵计算，同时还提供了高效的数据拷贝引擎TMA，它可以实现全局内存和共享内存之间异步高效数据搬运。这样便可以实现TMA完成全局内存到共享内存的加载，异步的Tensor Core利用WGMMA实现从共享内存读取数据进行矩阵运算，运算结束后将结果输出到全局内存。如图3所示，，同时Tensor Core循环的读取共享内存数据进行矩阵运算，将结果写入寄存器。可以看到异步的数据搬运单元TMA和计算单元Tensor Core通过中间介质共享内存完成数据依赖的解耦，逻辑上Tensor Core并不需要知道共享内存中数据是怎么获得的，其只需要知道其计算依赖的数据是不是就绪，如果Tensor Core所依赖的数据在共享内存中已经就绪，其就可以开始计算，如果共享内存中不间断的有数据就绪，则Tensor Core可以不间断的进行运算，达到Tensor Core的最大使用效率，同时针对异步的TMA而言，其也不需要关注共享内存的数据被谁消耗，其只需要关注有没有可以用来写入的共享内存空间，如果这块空间没有被依赖则其可以写入新的数据以供Tensor Core来使用，通过共享内存作为中间数据交换点，可以解耦数据加载和计算，使得单边点性能达到最高，同时使用共享内存作为中间缓冲区，可以更好的平衡由于数据加载抖动引起的计算性能波动。

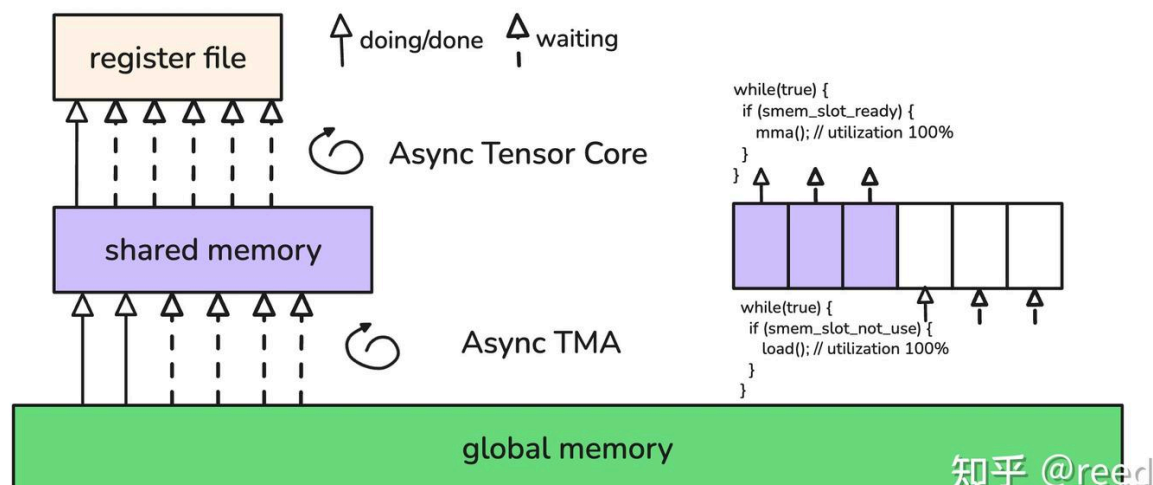


Figure-3. A typical GEMM data dependency and decoupled by shared memory

以上编程范式在传统的算法中表现为生产者消费者模型，在一个典型的C++实现中，其通过锁和条件变量实现，为了更好的适配这种编程范式，使得Tensor Core和TMA能够更好的发挥各自的性能，Hopper架构上提供了能够协调线程到达，数据到达和等待的高效的同步机制，即MBarrier。

传统的barrier（栅栏）在NVidia GPU体系上实现为独立的硬件单元，如常用的CUDA线程同步函数函数__syncthreads(),其有两条语义，第一是执行同步，第二是内存可见性屏障。对于执行同步而言，即所有参与线程在调用__syncthreads()后都会等待其他线程，只有当所有线程到达该同步点后才能继续执行后续的指令，即其可以表达为两个步骤，i. 线程到达，ii. 等待其他线程到达；对于可见性而言，__syncthreads()能够确保该调用之前的任意线程发起的共享内存写入操作对该调用之后的所有线程可见，借用C++内存模型表示，可以简单的表达为在线程同步之前调用内存release()语义，在同步之后施加acquire语义，简单的可以将__syncthreads()拆解如下

```
void __syncthreads() {
  memory_release();
  thread_arrive();
  wait_all_threads();
  memory_acquire();
}
```

除了上面的线程块（block）内所有线程都需要参与的同步机制__syncthreads(),CUDA还通过PTX提供了named barrier同步机制bar.sync a b; 其中a表示所name barrier的id，每一个线程块可以使用0到15共16个有名的barrier，b表示线程块内有多少线程需要参与。相较于

`__syncthreads()`的所有参与线程都需要同步，named barrier可以实现部分线程的同步，可以实现更精细的同步控制，其核心逻辑可以拆解如下：

```
void bar_sync(int id, int count) {  
    memory_release();  
    thread_arrive(id);  
    wait_threads(id, count);  
    memory_acquire();  
}
```

更进一步地，为了提高barrier的控制能力和细节，NVidia提出了mbarrier，它缩写自memory barrier，意为在内存上的barrier，这里的内存为共享内存，概念上它不再是有限个数的硬件单元，而是可以和共享内存的空间一样大（如图4所示）。它实现了更精细的语义控制，提供独立的arrive和wait能力，除此之外mbarrier还将TMA异步拷贝的完成机制也合并到其中，当TMA拷贝完成时能够改变该mbarrier的状态，这样使用统一的wait就可以等待TMA异步拷贝完成。

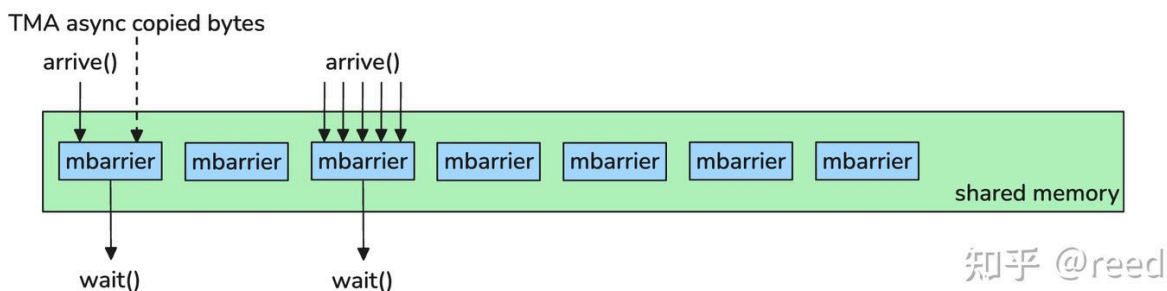


Figure-4. mbarrier semantic

mbarrier使用SM上的共享内存作为存储后端，为了提高其操作效率，硬件层面提供cache机构对其加速，只在初始化和销毁barrier的时候cache内容才会写回（write back）共享内存，其他操作均可以发生在cache内，并不向共享内存写回，所以虽然从指令层面看，barrier对应的SASS指令SYNCS操作对象为共享内存，但是其操作本身内并不用向共享内存写回，其效率要远高于共享内存操作。

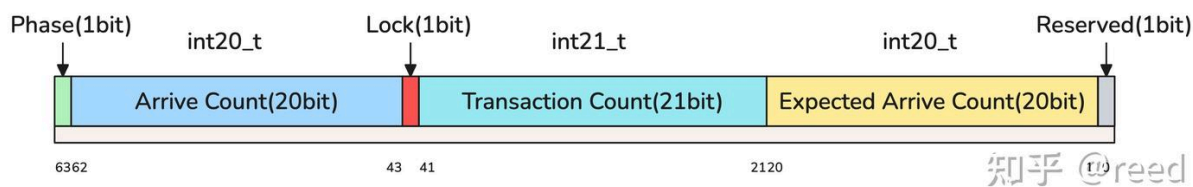


Figure-5. MBarrier fields

mbarrier在数据表示上表达为一个64bit的整数类型数据，其内部各个域的，整体分为六个域，分别是最低位的一比特的保留域，第一比特到第二十比特的Expected Arrive Count域，第二十一比特到第四十一比特的Transaction Count域，第四十二比特位置的Lock域，第四十三比特到第六十二比特的Arrive Count域和第六十三比特位置的Phase域。

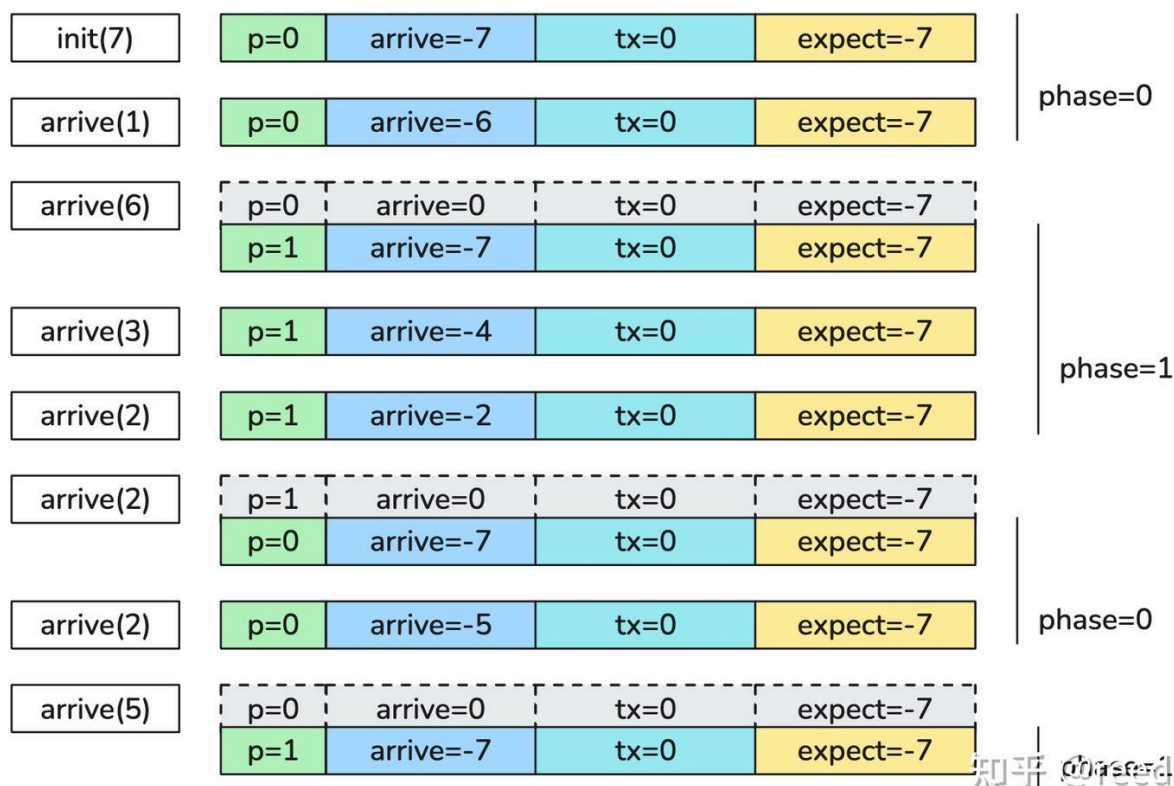


Figure-6. mbarrier state update with arrival

具体地，在mbarrier初始化的时候，Expected Arrive和Arrive Count域会被设置为负数表示的初始化数值，如初始化mbarrier为7，即等待7个到达的arrive，则Expected Arrive Count和Arrive Count都会被设置为-7，表示需要等待7个到达，并且这两个域都是二十比特，高位为符号位，表示为的有符号整数类型int20_t，表示正数使用原码，表示负数时使用补码，如-7表示为

b11111111111111111001，初始化时Transaction Count域会被设置为0，该域和Arrive Count域类似，也是有符号的数据，只是bit位数比其多一，整体表示为数据类型int21_t，初始化时Lock域会被设置为0，表示是一种正常的状态，Phase域在初始化的时候被设置为0表示当前为phase 0，其只有一比特表示，所以其是只有0 1两种状态，使用0 1两种状态来完成对mbarrier的状态维护和复用，具体的理论可以参考并行计算领域的Sense Reversing Barrier部分。如图6，当有线程调用arrive(n)的时候则Arrive Count域会加上n，当加上n之后如果Arrive Count域刚好等于0，则表示该mbarrier完成，则其phase自动切换到下一个状态（如由0切换到1，或由1切换到0），同时Arrive Count域自

动重置为Expected Arrive Count，表示在新的phase重新需要等待新的arrive到来。如果加上n之后不能到达0，则表示这次到达并不能完成该phase，则phase不变，只是Arrive Count加上n，需要等待后续的arrive到来才能完成该phase，如果一次arrive的数量n加上Arrive Count后大于0，则此mbarrier出错，Lock设置为1进入错误锁定状态。当phase切换后（如图中的灰色表示），对该mbarrier的wait条件是可以满足的，则调用wait(phase)线程可以继续，否则wait线程会等待Arrive Count到达0是才可以继续执行，值得注意的是初始化状态，即 $p=0, arrive=-7, tx=0, expect=-7$ ，也可以认为是由 $p=1, arrive=0, tx=0, expect=-7$ 切换而来的状态，所以在该状态进行wait(phase=1)是满足的。

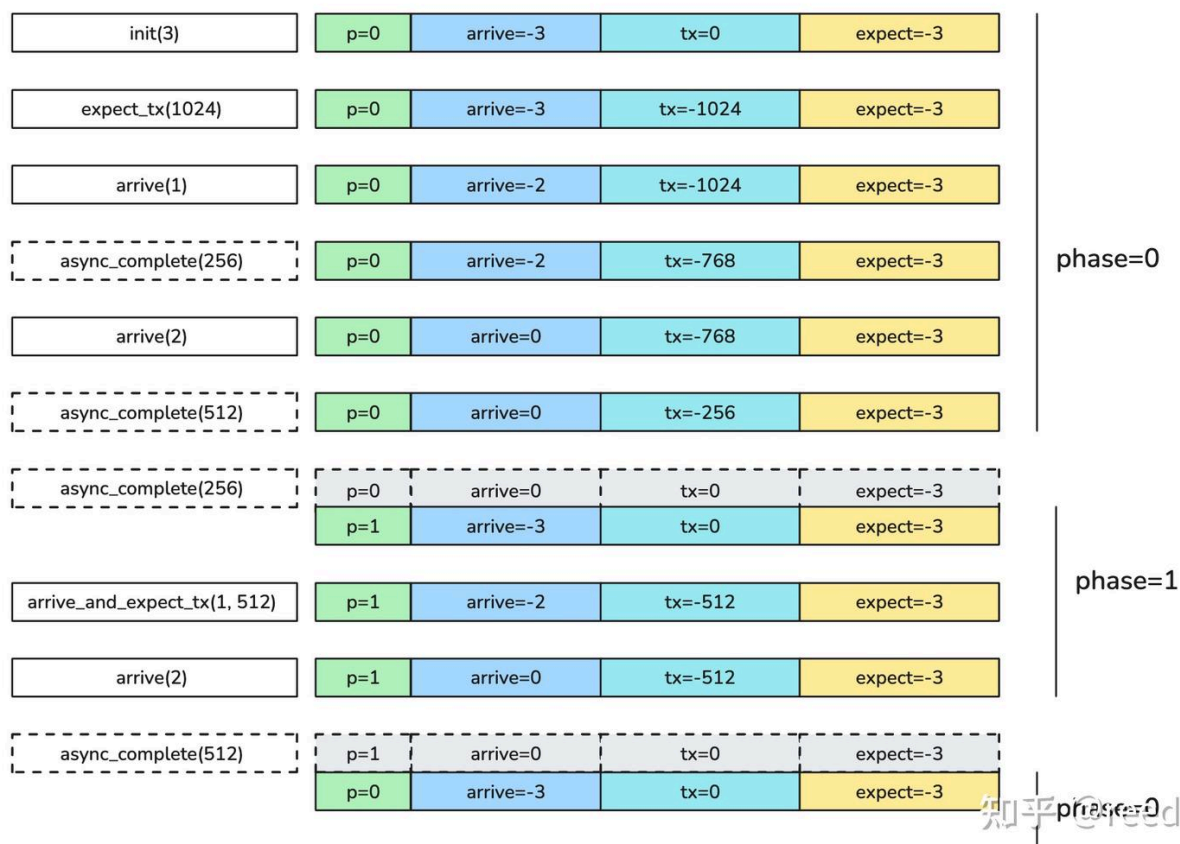


Figure-7. mbarrier state update with asynchronous transaction

以上介绍了mbarrier状态随着arrive事件各个field的更新过程，下面我们介绍由transaction count相关的状态转换，如图7，mbarrier初始化时设置为3，则arrive count和expected arrive count都会设置为-3表示需要等待3次arrive，同时transaction count (tx) 会被初始化为0，phase会被初始化为0。此时我们设置mbarrier需要等待的transaction bytes，如图示例，我们设置了1024bytes，设置后则tx更新为-1024，表示需要等待1024bytes数据的到来，和前面介绍的类似，我们可以通过arrive来完成到达，同时我们会把mbarrier设置给TMA的拷贝单元，当TMA完成拷贝时，其会自动更新mbarrier的tx，如图中虚线框async_complete所示，当TMA完成数据拷贝时，其会通知mbarrier数据到达，从而更新mbarrier的tx字段会加上到达的数据量，当tx到达后如果arrive字段和tx字段同时

到达0，则phase完成反转，Arrive Count字段会重置为Expect Arrive Count的值，同时Transaction Count域会被重置为0，这是可以对其进行新一轮等待数据进行设置，来进行对mbarrier对重新复用，图中的灰色框表示该phase的完成，此时对该mbarrier进行wait相应的phase则可以完成，如果在该位置之前进行等待则需要等arrive和tx事件完成方可等待完成，在phase反转之后对上一phase进行等待也可以立即完成。mbarrier同时提供了指令能够同时指定arrive和tx，其能原子的实现对该数据的更新，减少操作mbarrier的指令调用次数。除了对mbarrier直接进行wait，还可以通过PTX提供对test指令来实现对mbarrier状态的探测。

mbarrier提供的wait可以指定内存可见性scope，这样当wait成功时，可以确保整个scope是可以看见其内存副作用的，避免了需要所有线程wait。

CUDA以PTX的形式提供了mbarrier的能力，cute针对mbarrier进行了函数封装（位于cute/arch/copy_sm90_desc.hpp），常用的有

```
void
initialize_barrier(uint64_t& smem_barrier,           // 64 bits user-
managed barrier in smem
                  int thread_count = 1)             // Thread count
expected to arrive/wait on this barrier
void
set_barrier_transaction_bytes(uint64_t& smem_barrier, // 64 bits user-
managed barrier in smem
                              uint32_t bytes)         // Number of bytes
transferred by per TMA transaction
void
wait_barrier(uint64_t& smem_barrier,                 // 64 bits user-
managed barrier in smem
             int phase_bit)                           // Current phase
bit the barrier waiting to flip
void
arrive_barrier(uint64_t& smem_barrier)                // 64 bits user-
manged barrier in smem
```

还有一部分能力被封装在了cutlass/arch/barrier.h中。

使用示例

针对以上mbarrier的状态转移过程，通过代码进行了验证，由于通常情况下mbarrier是被缓存在cache中的，并且不会写回共享内存，正常情况下我们无法通过读取共享内存来获取mbarrier的数据，但我们可以通过调用**mbarrier.inval**指令来销毁mbarrier，这会使得cache中的数据向共享内存写回，这时我们就可以通过读取共享内存（LDS）来获取mbarrier的状态了。具体实验代码参见<https://github.com/reed-lau/cute-gemm/tree/main/mbarrier>。

总结

本文介绍了Hopper上硬件加速的事务型内存同步机制MBarrier，重点介绍了其核心的数据表示和状态转移，它作为Hopper上高效的同步机制是实现生产者-消费者模式的必要组件，是TMA和Tensor Core形成流水线的核心组件，同时本文通过代码示例展示了mbarrier的状态转移过程。

参考

<https://patents.google.com/patent/US20230289242A1/en>

<https://docs.nvidia.com/cuda/parallel-thread-execution/#parallel-synchronization-and-communication-instructions-bar>

<https://mattchung.me/blog/2020/09/18/making-sense-of-the-sense-reversing-barrier-synchronization/>

https://en.cppreference.com/w/cpp/atomic/memory_order.html